ANKARA ÜNİVERSİTESİ

BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

3522 BULUT BİLİŞİM DERSİ

PRŐJE 1 RAPORU

ÇİFT KATMANLI WEB UYGULAMASI

(Web API + Frontend)

Hazırlayan:

SALİM CAN ŞEN

20291313

Ders Sorumlusu:

ENVER BAĞCI

Tarih:

Nisan 2026

1. Proje Özeti ve Temel Amaçlar

Günümüz dijital yaşantısında "zaman ve iş yönetimi" en temel ihtiyaçlardan biridir. Bu projenin çekirdek fikri, sadece standart bir "Yapılacaklar Listesi" üretmek değil; bu fikri endüstri standartlarına uygun bir iki katmanlı (Two-Tier) bulut bilişim mimarisi üzerinden modellemektir.

Bu doğrultuda projede şu hedefler başarıyla gerçekleştirilmiştir:

- İzole İstemci-Sunucu Yapısı: Sunum katmanı (Frontend) ile Veri/İş mantığı katmanı (Backend) birbirinden tamamen ayrılarak "Stateless" bir API entegrasyonu sağlandı.
- Kapsamlı Görev Yönetimi: Kullanıcılara görev önceliği (Priority), son teslim tarihi (Deadline) ve anlık tamamlanma oranı gibi detaylı izleme imkanları sunuldu.
- Kusursuz UI/UX Deneyimi: Glassmorphism (cam tasarımı) prensipleriyle modern bir arayüz oluşturuldu; arama ve sıralama gibi işlemler sayfa yenilenmeden (SPA) anında gerçekleştirildi.

2. Sistem Mimarisi ve Kullanılan Teknolojiler

Proje, bulut ortamında dağıtıma (deployment) hazır, birbirinden bağımsız çalışan iki ana katmandan oluşmaktadır.

Kullanılan Teknolojiler

- Backend (Katman 1): Node.js (v18+) çalışma ortamında, Express.js (^4.18) ile RESTful API yönlendirmeleri sağlandı. Çapraz port istekleri için cors, ortam değişkenleri için dotenv kullanıldı.
- Veri Katmanı: Hafifliği ve sunucusuz çalışabilmesi sebebiyle SQLite (better-sqlite3) tercih edildi.

- Frontend (Katman 2): React.js (^18.x) ve Vite altyapısıyla tek sayfa uygulaması (SPA) inşa edildi. Backend ile iletişim Axios kütüphanesi üzerinden kuruldu.

Mimarinin Şematik Gösterimi

Aşağıdaki yapı, sistemin port bazlı izolasyonunu ve veri akışını göstermektedir:

```
┌──────────────────────────────────────┐
│        FRONTEND (React.js)           │  ← Port 3000
│  - Kullanıcı Arayüzü (SPA)           │
│  - Axios HTTP İstekleri              │
│  - State Management                  │
└──────────────────────────────────────┘
                │
                │  HTTP / REST API
                │  (JSON formatında)
                ▼
┌──────────────────────────────────────┐
│     BACKEND (Node.js/Express)        │  ← Port 5000
│  - RESTful API Endpoints             │
│  - İş Mantığı (Controller)           │
│  - CORS, dotenv, Middleware          │
└──────────────────────────────────────┘
                │
                │  SQL Sorguları
                │  (better-sqlite3)
                ▼
┌──────────────────────────────────────┐
│         VERİTABANI (SQLite)          │
│  - todos tablosu                     │
│  - Yerel .db dosyası                 │
└──────────────────────────────────────┘
```

3. Veritabanı Modeli ve Geliştirme Süreci

Geliştirme süreci veritabanı modellemesiyle başlamış, API entegrasyonu ve ardından UI inşasıyla devam etmiştir. Sistemdeki tüm görevler, gelişmiş parametreler barındıran tekil bir todos tablosunda tutulmaktadır.

SQL Tablo Şeması:

```sql
CREATE TABLE IF NOT EXISTS todos (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  title       TEXT    NOT NULL,
  description TEXT    DEFAULT '',
  completed   INTEGER DEFAULT 0,         -- 0: bekliyor, 1: tamamlandı
  priority    TEXT    DEFAULT 'medium',  -- low | medium | high
  due_date    TEXT,                      -- Son teslim tarihi
  created_at  TEXT    DEFAULT (datetime('now', 'localtime')),
  updated_at  TEXT    DEFAULT (datetime('now', 'localtime'))
);
```

Geliştirme Adımları:

- Model Katmanı: SQLite veritabanı kuruldu ve CRUD operasyonları için senkron sorgular yazıldı.
- Controller ve Route Katmanı: İstemciden gelen HTTP isteklerini karşılayacak iş mantıkları (Controller) ve endpoint'ler (Route) express.Router() ile modüler hale getirildi.
- Arayüz Entegrasyonu: React State yapısı kurularak useEffect üzerinden veriler anlık olarak ekrana basıldı.

4. Karşılaşılan Mimari Sorunlar ve Çözüm Teknikleri

Bu çapta dağıtık bir sistem kodlanırken karşılaşılan temel mühendislik problemleri ve çözümleri şunlardır:

- CORS Politikaları ve Port Çarpışması:
  - Sorun: React (Port 3000) ve Node.js'in (Port 5000) farklı portlarda çalışması, tarayıcıların Preflight isteklerini engellemesine yol açtı.
  - Çözüm: Node.js tarafında bir Middleware katmanı oluşturularak app.use(cors({ origin: '*' })) izni entegre edildi ve iki izole sunucunun güvenli iletişimi sağlandı.

- Veritabanı Göçü (Database Migration) Riski:
  - Sorun: Projeye sonradan due_date özelliği eklendiğinde, aktif verilerle dolu SQLite tablosunu CREATE TABLE ile ezmek veri kaybına yol açacaktı.
  - Çözüm: Kodlara try-catch mekanizması eklenerek güvenli bir yama sorgusu (Alter Table) çalıştırıldı:

```
ALTER TABLE todos ADD COLUMN due_date TEXT;
```

- State Karmaşası ve Performans:
  - Sorun: Arama, filtreleme ve sıralama işlemleri için sürekli Backend'e istek atmak, uygulamada gecikmelere (lag) sebep oldu.
  - Çözüm: Optimizasyona gidilerek verilerin sadece açılışta çekilmesi sağlandı. Tüm filtreleme işlemleri Backend'i yormadan, doğrudan Frontend üzerinde .filter() ve .sort() metodlarıyla milisaniyeler içinde gerçekleştirildi.

5. Bulut Dağıtım Stratejisi ve AWS Entegrasyonu

Projenin yerel ortamdan buluta taşınma sürecinde yüksek erişilebilirlik sağlamak amacıyla AWS EC2 (Elastic Compute Cloud) sanal sunucu altyapısı tercih edilmiştir. Sistem, iki katmanlı mimarisine uygun olarak bulut üzerinde de tamamen izole portlar üzerinden haberleşmektedir.

Bu kapsamda uygulanan bulut ve ağ (network) yapılandırmaları şunlardır:

- Port 3000 (Frontend Katmanı): React.js arayüzünün dış dünyaya servis edildiği ve istemci bağlantılarının karşılandığı ana porttur.
- Port 5000 (Backend Katmanı): Node.js/Express REST API sunucusunun dinlediği ve veritabanı işlemlerini yürüttüğü izole porttur.
- Güvenlik Grupları (Security Groups): AWS Console üzerinden yapılandırılan "Inbound Rules" ile dışarıdan yalnızca yetkilendirilmiş HTTP/HTTPS (80/443) ve uygulamanın özel portlarına (3000, 5000) erişim izni verilerek ağ güvenliği sağlanmıştır.
- Nginx Ters Vekil (Reverse Proxy): AWS EC2 sunucusuna gelen dış trafik Nginx ile karşılanmış, gelen istekler arka plandaki ilgili portlara güvenli bir şekilde yönlendirilerek port karmaşası önlenmiştir.

Bu dağıtım mimarisi; projenin gelecekte Docker konteynerleri ve AWS RDS (PostgreSQL) gibi tam teşekküllü bulut servisleriyle kolayca ölçeklenebilmesi için sağlam bir zemin hazırlamıştır.